Huffman Coding-Greedy Method

Dr. M. Nalliah
Associate Professor
School of Computer Science and Engineering,
Vellore Institute of Technology
Vellore-632014. *nalliah.moviri@vit.ac.in*

March 26, 2026

Outline

1. Huffman Encoding
2. Encoding Types
3. Huffman Encoding

Feasible, Objective, and Optimal Solutions

- **Feasible Solution:** Any subset that satisfies the given constraints is called a feasible solution.
- **Objective Function:** Any feasible solution needs either to maximize or minimize a given function, which is called the objective function.
- **Optimal Solution:** Any feasible solution that maximizes or minimizes the given objective function is called an optimal solution.

Greedy Method

- Greedy method constructs a solution to an optimization problem piece by piece through a sequence of choices that are:
  - **Feasible** – satisfying the constraints.
  - **Locally optimal** – the best local choice among all feasible choices available at that step.
  - **Irrevocable** – once made, it cannot be changed on subsequent steps of the algorithm.
- For some problems, it yields a globally optimal solution for every instance.
- The following is the general greedy approach for control abstraction of subset paradigm.

Greedy Algorithm

The following is the general greedy approach for control abstraction of subset paradigm.

```
Algorithm Greedy(a,n)
//a[1:n] contains n inputs
{
  solution := Φ // initializes to empty
  for i:=1 to n do
  {
    x := select(a);
    if Feasible(solution x) then
       solution := union(solution, x)
  }
  return solution;
}
```

Greedy Example: Minimum Coin Change

- Problem: Given unlimited amounts of coins of denominations $d_1 > \cdots > d_m$, give change for amount n with the least number of coins.
- Example: $d_1 = 25c$, $d_2 = 10c$, $d_3 = 5c$, $d_4 = 1c$, and $n = 48c$.
- **Greedy Approach:** At each step, take the maximum denomination coin $\leq$ remaining amount.
- Steps:
  1. $48 - 25 = 23$
  2. $23 - 10 = 13$
  3. $13 - 10 = 03$
  4. $03 - 01 = 02$
  5. $02 - 01 = 01$
  6. $01 - 01 = 00$
- **Solution:** $\langle 1, 2, 0, 3 \rangle$ i.e., d_1: 1 coin, d_2: 2 coins, d_3: 0 coins, d_4: 3 coins.
- Observation: Greedy solution is optimal for any amount and "normal" set of denominations.

Definition of Encoding

- Encoding is the process of transmitting or storing sequences of characters efficiently.
- In computer science, every piece of information is represented as strings of 1s and 0s.

Encoding Types

Fixed-Length Encoding

- Each symbol is represented using the same number of bits.
- Simple but may waste space if symbol frequencies are uneven.
- Example: ASCII encoding (7 or 8 bits per character).

Variable-Length Encoding

- Symbols are represented using different numbers of bits.
- Frequently used symbols get shorter codes.
- More efficient than fixed-length encoding.

Huffman Encoding

- A widely used variable-length encoding scheme.
- Based on symbol frequency.
- Produces optimal prefix codes (no code is a prefix of another).
- Forms the basis of many compression algorithms.

Fixed-Length Encoding

- Every character is assigned a binary code using the same number of bits.
- Simple but may waste space if symbol frequencies are uneven.
- Example: ASCII encoding (7 or 8 bits per character).
- For the string "aabacdad":
  - Each character uses 8 bits.
  - Total = 8 characters $\times$ 8 bits = 64 bits (8 bytes).

Variable-Length Encoding

- Characters are encoded using variable numbers of bits depending on their frequency.
- Frequently occurring characters get shorter codes, while rare ones get longer codes.
- More efficient than fixed-length encoding.
- For the string "aabacdad":
  - Character frequencies: a(4), b(1), c(1), d(2).
  - Since 'a' occurs most frequently, it is assigned the shortest code.
  - Example code assignment:

Character	Code
a	0
b	011
c	111
d	11

  - Encoded string: 00011011111011
  - Breakdown: (0 — 0 — 011 — 0 — 111 — 11 — 0 — 11)
  - Uses fewer bits compared to fixed-length encoding.

Problem with Decoding

- The real challenge lies in the decoding phase.
- If we try to decode the string `00011011111011` (for "aabacdad"), it becomes ambiguous.
- Multiple possible decodings exist:
  - `aaadacdad` (0 — 0 — 0 — 11 — 0 — 111 — 11 — 0 — 11)
  - `aaadbcad` (0 — 0 — 0 — 11 — 011 — 111 — 0 — 11)
  - `aabbcb` (0 — 0 — 011 — 011 — 111 — 011)
  - ...and many more
- This ambiguity arises because some codes are prefixes of others.
- Solution: Use **prefix-free codes** (e.g., Huffman Encoding) to ensure unique decodability.

Prefix Rule

- To prevent ambiguities during decoding, the encoding phase must satisfy the **prefix rule**.
- Prefix Rule: No binary code should be a prefix of another code.
- This ensures that codes are **uniquely decodable**.
- If the rule is broken, multiple interpretations may appear during decoding.
- Example: In the earlier assignment,
  - Code for ‘a‘ = 0
  - Code for ‘b‘ = 011
  - Here, 0 is a prefix of 011 $\Rightarrow$ ambiguity arises.

Corrected Encoding with Prefix Rule

- Let us reconsider assigning codes to characters ‘a‘, ‘b‘, ‘c‘, and ‘d‘:

Character	Code
a	0
b	11
c	101
d	100

- These codes satisfy the prefix rule (no code is a prefix of another).
- Encoding the string “aabacdad”:
  - Encoded string: `001101011000100`
  - Breakdown: (0 — 0 — 11 — 0 — 101 — 100 — 0 — 100)
  - Decoding is now unambiguous: `aabacdad`

Huffman Encoding

- Developed by **David Huffman** in 1951.
- Basis for many data compression and encoding schemes.
- Famous algorithm used for **lossless data compression**.
- Follows a **Greedy approach**:
  - Generates minimum-length prefix-free binary codes.
- Uses a **variable-length encoding scheme**:
  - Characters occurring more frequently are assigned shorter codes.
  - Less frequent characters are assigned longer codes.
- Ensures unique decodability by satisfying the **prefix rule**.

Huffman Coding as a Greedy Algorithm

- **Huffman Coding** is a well-known Greedy algorithm.
- The size of the code allocated to a character depends on its frequency.
- Reason it is called **Greedy**:
  - The character with the highest frequency is assigned the shortest code.
  - Characters with lower frequencies are assigned longer codes.
- Employs a **variable-length encoding scheme**:
  - Each character in the data stream receives a different variable-length code.
  - Ensures efficient compression while maintaining unique decodability.
- Forms the foundation of many lossless compression techniques.

In fixed-length encoding, all the letters/symbols are represented using an equal number of bits.

Encoding for each symbol				Encoding for "ABBA"
A	B	C	D	
00	01	10	11	00 01 01 00
01	1	000	001	01 1 1 01
0	1	10	11	0 1 1 0

Fixed length

Example message BCCABBDDAECCBBAEDDCC 8bits *20chars =160 bits

Character	Count/Frequency	Code
A	3	000
B	5	001
C	6	010
D	4	011
E	2	100

message = 20*3 =60 bits
Table =40(ABCDE-each one 8 bits(5(ABCDE)×8))+15(mapping to 3bits,5(ABCDE)×3)
=55 bits
Total = 115 bits

Compression Example

- Example message: `BCCABBDDAECCBBAEDDCC`
- **Fixed-Length Encoding:**
  - Each character uses 8 bits.
  - Message length = 20 characters $\times$ 8 bits = 160 bits.
- **Variable-Length Encoding:**
  - Each character uses on average 3 bits.
  - Message length = 20 characters $\times$ 3 bits = 60 bits.
  - Encoding Table size = 40 bits (for symbols A–E) + 15 bits (for codes) = 55 bits.
  - Total size = 60 + 55 = 115 bits.

Huffman Codes

- (i) Data can be encoded efficiently using Huffman Codes.
- (ii) It is a widely used and beneficial technique for compressing data.
- (iii) Huffman's greedy algorithm uses a table of the frequencies of occurrences of each character to build up an optimal way of representing each character as a binary string.

This greedy algorithm finds an optimal way to encode each character as a binary string by evaluating the frequencies of characters and using a frequency-sorted binary tree. Characters with higher occurrences are encoded as shorter-bit strings while those that appear less frequently are encoded as longer-bit strings.

- Huffman Coding is a famous Greedy Algorithm.
- It is used for the lossless compression of data.
- It uses variable length encoding.
- It assigns variable length code to all the characters.
- The code length of a character depends on how frequently it occurs in the given text.
- The character which occurs most frequently gets the smallest code.
- The character which occurs least frequently gets the largest code.
- It is also known as Huffman Encoding

Major Steps in Variable Length Huffman Coding

There are two major steps in Huffman Coding

- Building a Huffman Tree from the input characters.
- Assigning code to the characters by traversing the Huffman Tree.

Huffman Tree:The steps involved in the construction of Huffman Tree are as follows-

- Step-01: Create a leaf node for each character of the text. Leaf node of a character contains the occurring frequency of that character.
- Step-02: Arrange all the nodes in increasing order of their frequency value.
- Step-03:
  Considering the first two nodes having minimum frequency,
  Create a new internal node. The frequency of this new node is the sum of frequency of those two nodes. Make the first node as a left child and the other node as a right child of the newly created node.

- Step-04:
  Keep repeating Step-02 and Step-03 until all the nodes form a single tree. The tree finally obtained is the desired Huffman Tree.

Problem: A file contains the following characters with the frequencies as shown. If Huffman Coding is used for data compression,

Characters	Frequencies
a	10
e	15
i	12
o	3
u	4
s	13
t	1

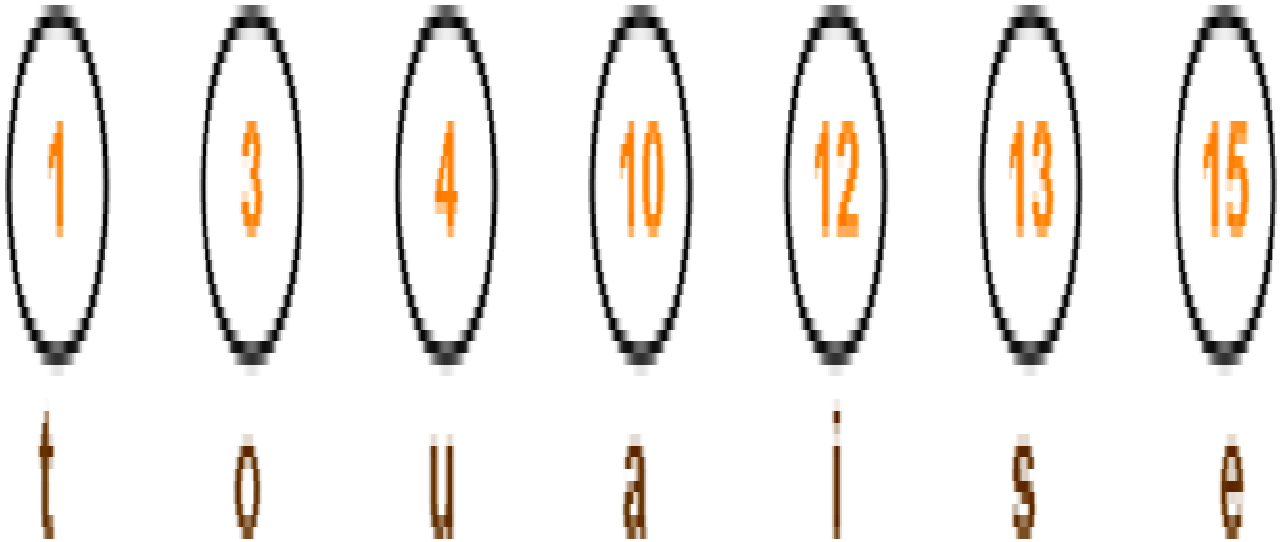
Problem cont...

Determine

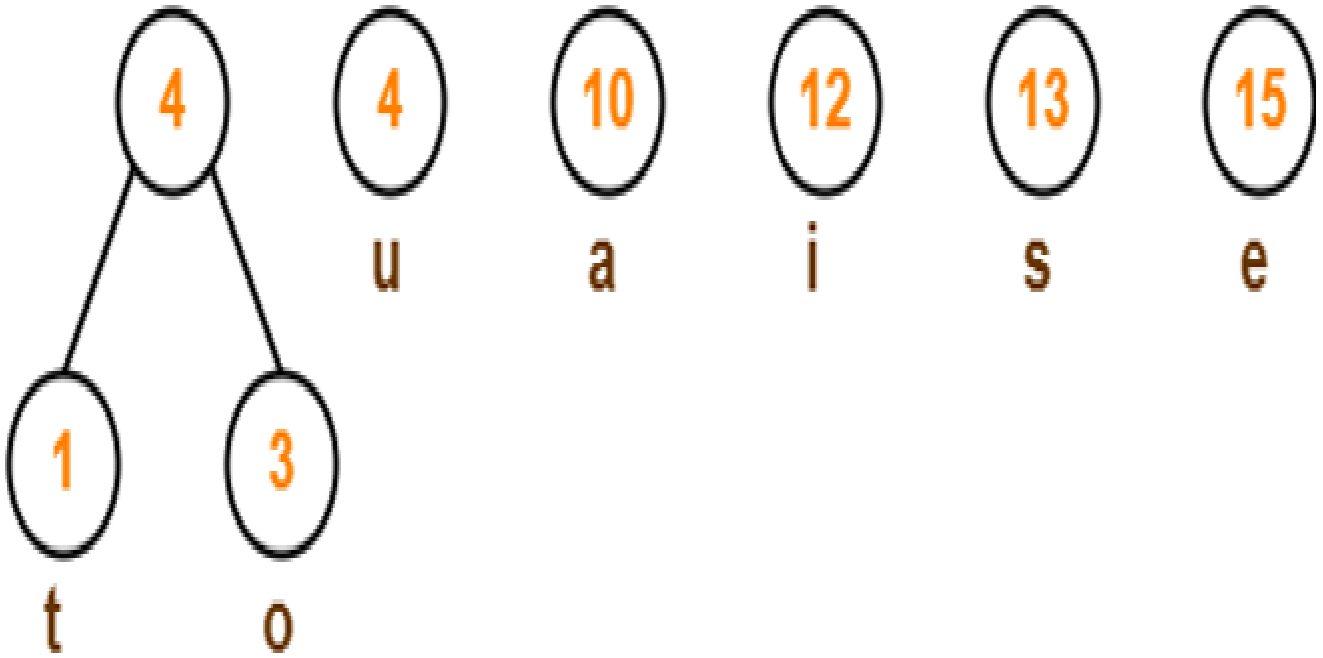
- Huffman Code for each character
- Average code length
- Length of Huffman encoded message (in bits)

Solution: First let us construct the Huffman Tree. Huffman Tree is constructed in the following steps.

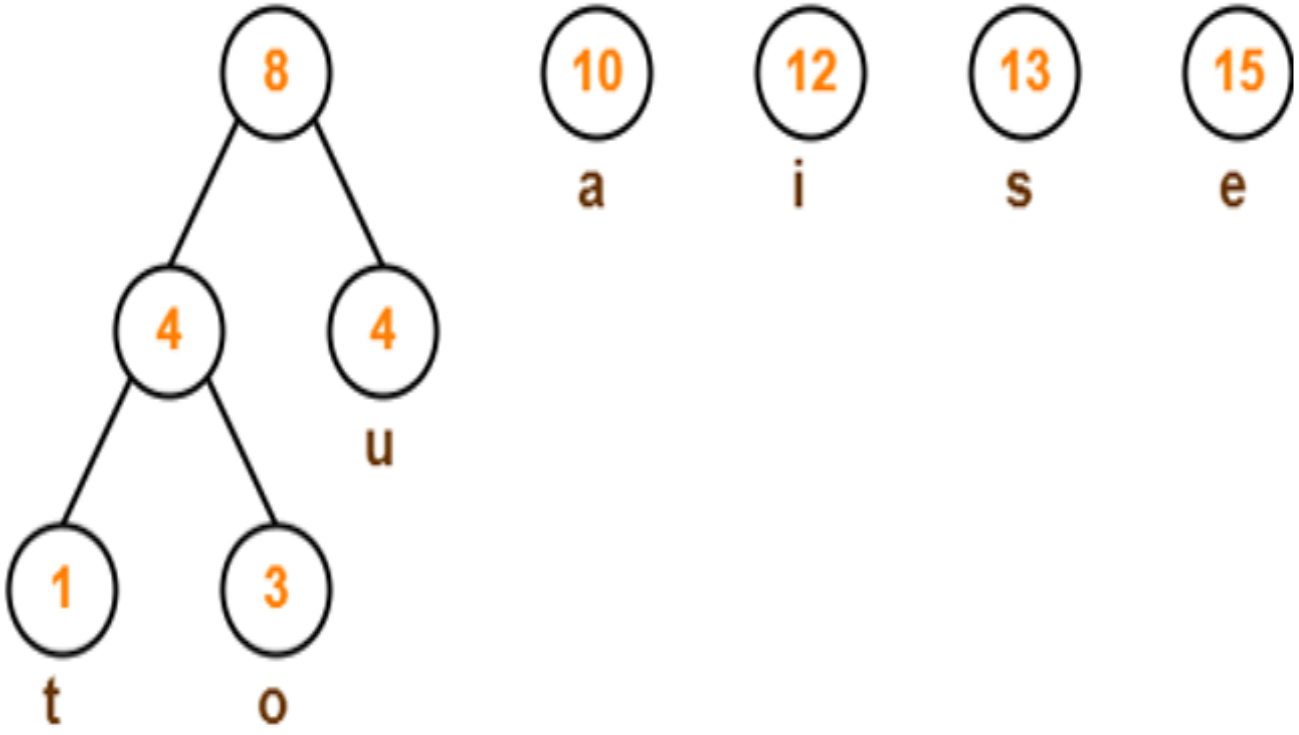
Step-01:



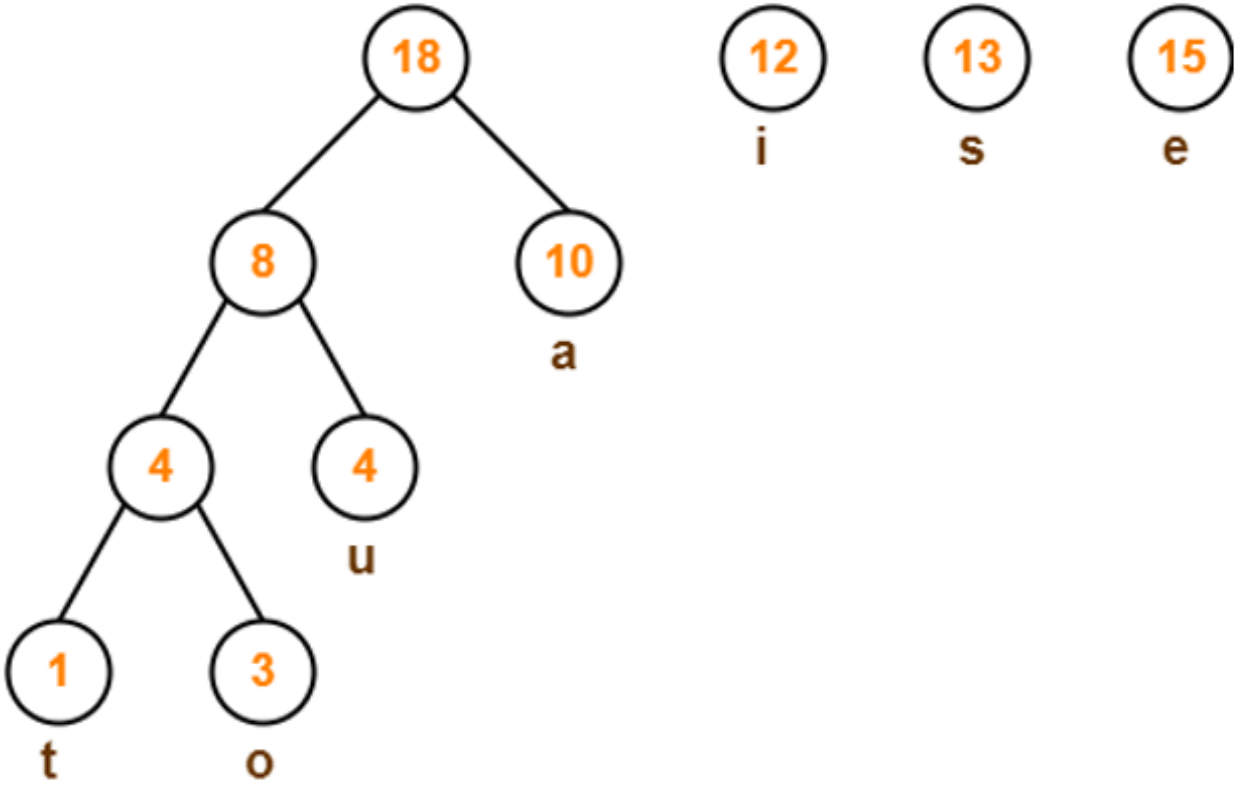
Step-02:



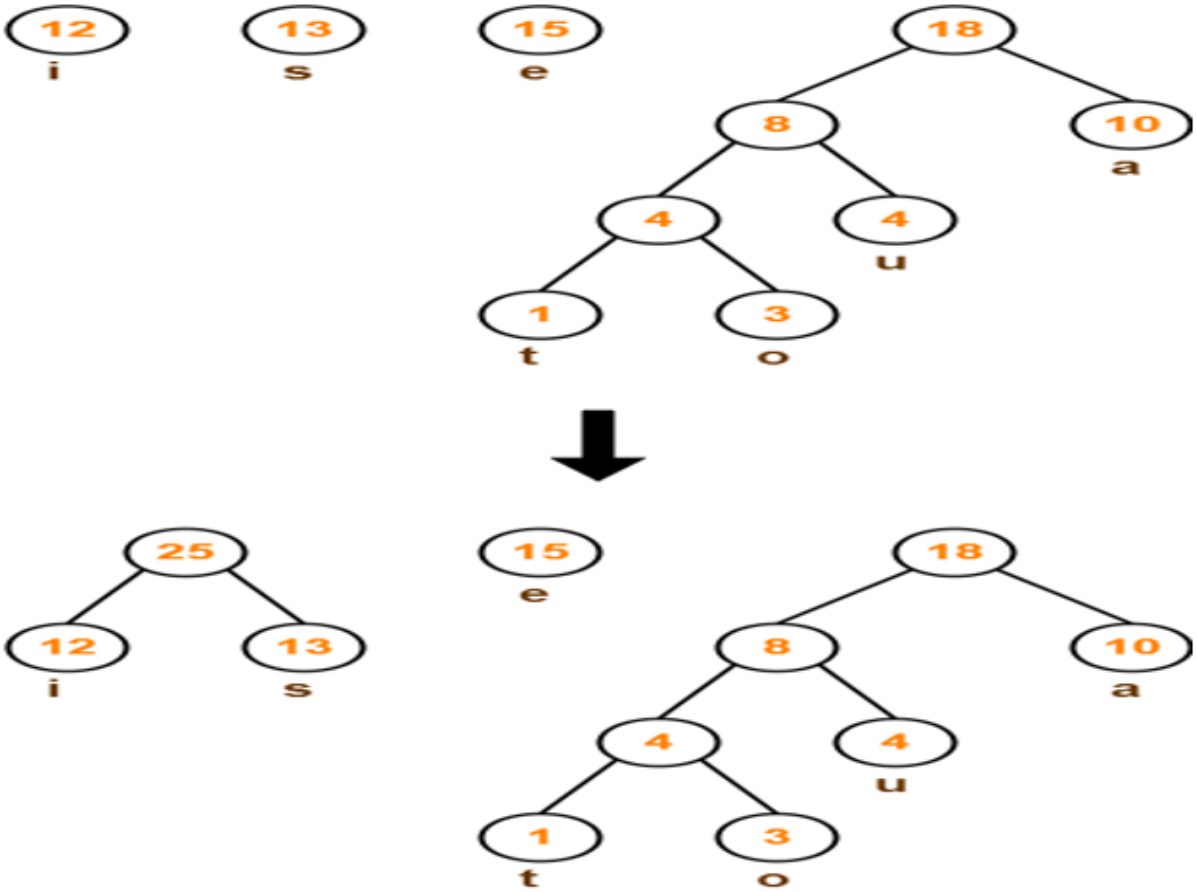
Step-03:



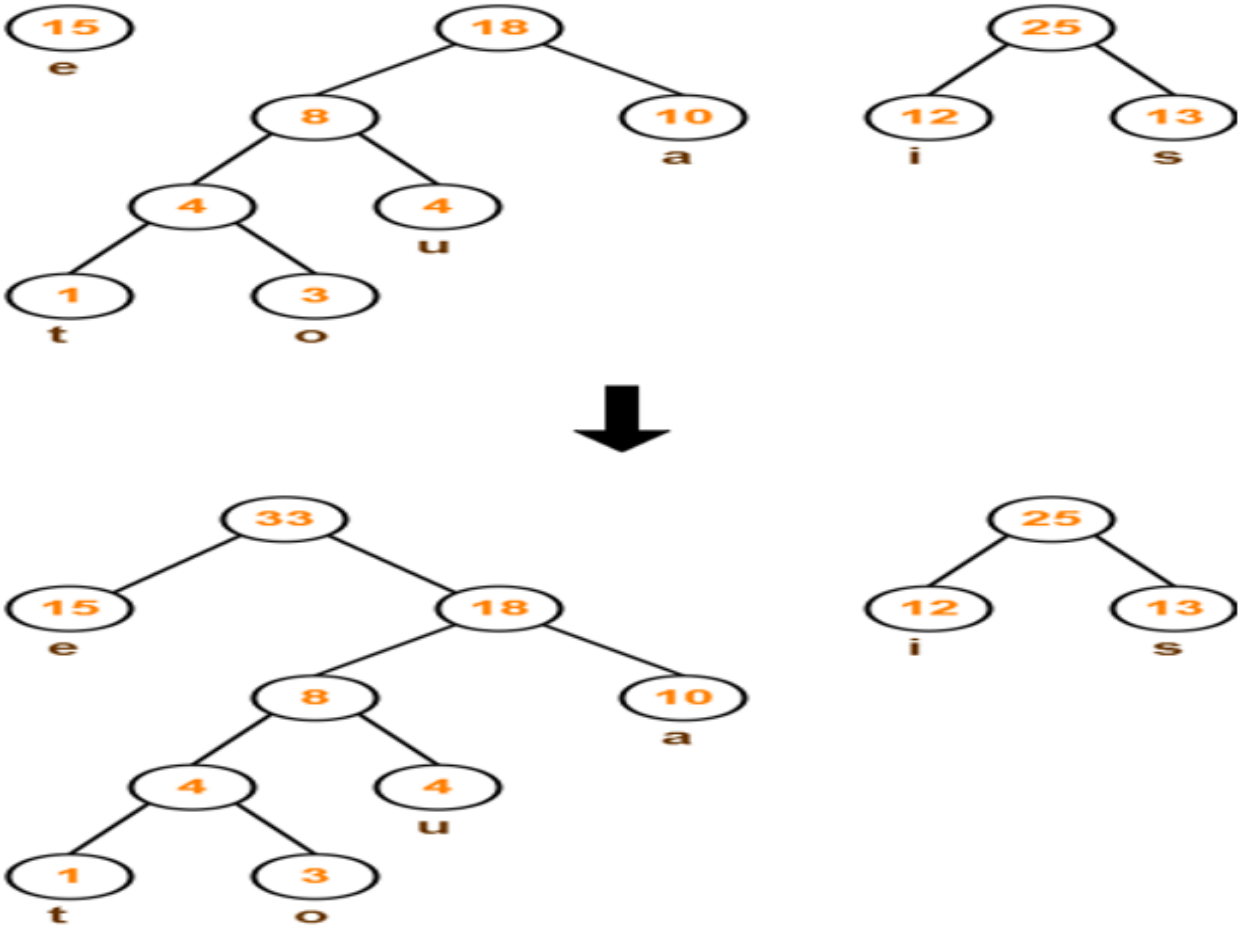
Step-04:



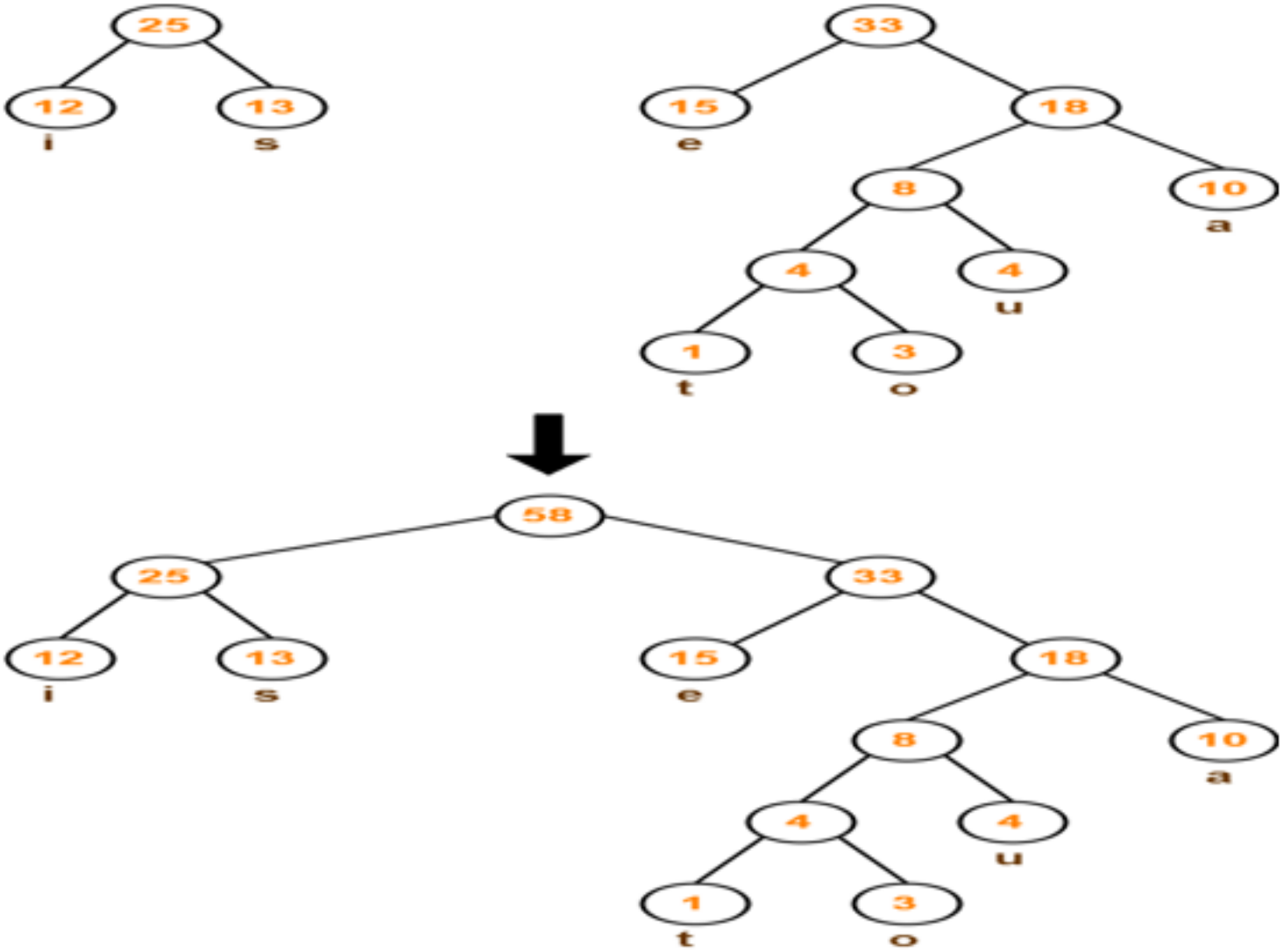
Step-05:



Step-06:

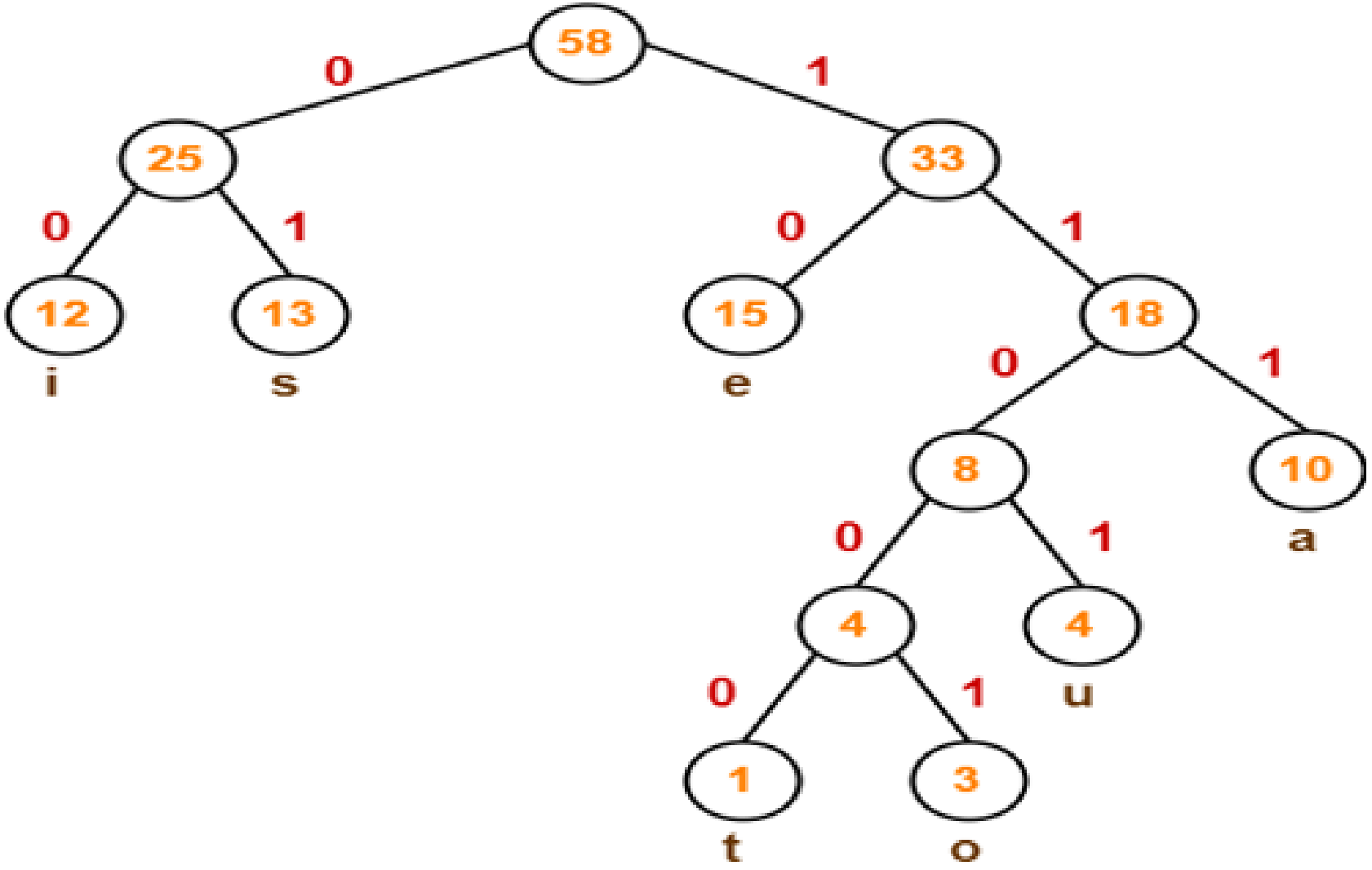

15
e
18
8
10
a
4
4
u
1
t
3
o
25
12
i
13
s
33
15
e
18
8
10
a
4
4
u
1
t
3
o
25
12
i
13
s

Huffman Tree



Huffman Tree

Now,We assign weight to all the edges of the constructed Huffman Tree. Let us assign weight '0' to the left edges and weight '1' to the right edges.



Huffman Tree

Huffman Code for Characters

- To write Huffman Code for any character, traverse the Huffman Tree from the root node to the leaf node of that character.
- Following this rule, the Huffman Code for each character is:

Character	Huffman Code
a	111
e	10
i	00
o	11001
u	1101
s	01
t	11000

- Observations:
  - Characters occurring less frequently in the text are assigned larger codes.
  - Characters occurring more frequently in the text are assigned smaller codes.

Average Code Length

we have: Average code length $= \frac{\sum(\text{frequency}_i \times \text{code length}_i)}{\sum(\text{frequency}_i)}$

$=(10 \times 3) + (15 \times 2) + (12 \times 2) + (3 \times 5) + (4 \times 4) + (13 \times 2) +$
$(1 \times 5)/(10 + 15 + 12 + 3 + 4 + 13 + 1)$

$=(30 + 30 + 24 + 15 + 16 + 26 + 5)/58$

$=146/58=2.5$

Then the total number of bits in Huffman encoded message=
Total number of characters in the message $\times$
Average code length per character
$= 58 \times 2.52 = 146.16 = 147 bits$

Huffman Coding Algorithm

- Sort the characters by frequency, ascending. These are kept in a Q/min-heap priority queue.
- For each distinct character and its frequency in the data stream, create a leaf node.
- Remove the two nodes with the two lowest frequencies from the nodes, and the new root of the tree is created using the sum of these frequencies.
- Make the first extracted node its left child and the second extracted node its right child while extracting the nodes with the lowest frequency from the min-heap.
- To the min-heap, add this node.
- Since the left side of the root should always contain the minimum frequency.
- Repeat steps 3 and 4 until there is only one node left on the heap, or all characters are represented by nodes in the tree.
- The tree is finished when just the root node remains.

Time Complexity Analysis

- Huffman coding uses a **min-heap** data structure for implementing the priority queue.
- The overall time complexity is **O(n log n)**.
- Explanation:
  - Building a min-heap takes **O(n log n)** time.
  - Moving an element from root to leaf requires **O(log n)** comparisons.
  - This is done for **n/2 elements** in the worst case.
  - Extracting the minimum frequency from the priority queue occurs **2(n - 1)** times, each with **O(log n)** complexity.
- Therefore, the total complexity remains **O(n log n)**.
- Note: A **linear time algorithm** is possible if characters are already sorted by frequency.

Advantages of Huffman Encoding

- This encoding scheme results in saving lot of storage space, since the binary codes generated are variable in length
- It generates shorter binary codes for encoding symbols/characters that appear more frequently in the input string
- The binary codes generated are prefix-free.

Disadvantages of Huffman Encoding

- Lossless data encoding schemes, like Huffman encoding, achieve a lower compression ratio compared to lossy encoding techniques. Thus, lossless techniques like Huffman encoding are suitable only for encoding text and program files and are unsuitable for encoding digital images.
- Huffman encoding is a relatively slower process since it uses two passes- one for building the statistical model and another for encoding. Thus, the lossless techniques that use Huffman encoding are considerably slower than others.
- Since length of all the binary codes is different, it becomes difficult for the decoding software to detect whether the encoded data is corrupted. This can result in an incorrect decoding and subsequently, a wrong output.

Time and Space Complexity

- **Time Complexity:** O(n log n), where n is the number of unique characters.
- Explanation:
  - If there are n nodes, `extractMin()` is called $2(n-1)$ times.
  - Each `extractMin()` takes O(log n) time as it calls `minHeapify()`.
  - Thus, the overall complexity is O(n log n).
- Note: If the input array is already sorted, there exists a **linear time algorithm**.
- **Space Complexity:** O(n).

Tutorial Problem

How Can a Huffman Tree Be Constructed from Input Characters?

The frequency of each character in the provided string must first be determined.

Character	Frequency
a	4
b	7
c	3
d	2
e	4